

All

Q 1

Q 2

Q 3

Q 4

Q3. Pick from both sides!

Solved

Stuck somewhere?

Ask for help from a TA and get it resolved.

Get help from TA.

Problem Description

You are given an integer array **A** of size **N**.
You have to pick **B** elements in total. Some (possibly 0) elements from left end of array **A** and some (possibly 0) from the right end of array **A** to get the maximum sum.
Find and return this **maximum possible sum**.
NOTE: Suppose B = 4, and array A contains 10 elements, then

- You can pick the first four elements or can pick the last four elements, or can pick 1 from front and 3 from the back, etc. You need to return the maximum possible sum of elements you can pick.

Problem Constraints

1 <= N <= 10⁵
1 <= B <= N
-10³ <= A[i] <= 10³

Input Format

First argument is an integer array **A**.
Second argument is an integer **B**.

Output Format

Return an integer denoting the maximum possible sum of elements you picked.

Example Input

Input 1:
A = [5, -2, 3 , 1, 2]
B = 3
Input 2:
A = [1, 2]
B = 1

Example Output

Output 1:
8
Output 2:
2

Example Explanation

Explanation 1:
Pick element 5 from front and element (1, 2) from back so we get 5 + 1 + 2 = 8
Explanation 2:
Pick element 2 from end as this is the maximum we can get

See Expected Output

Java7 (Open-Jdk-1.7.0)

```
44 |
45 |         sum = prefixSum
46 |     }else{
47 |         sum = prefixSum[i - 1] + p
48 |     }
49 |
50 |         if(max < sum)
51 |         {
52 |             max = sum;
53 |         }
54 |     }
55 |
56 |     return max;
57 | }
```

Test Output

Code's All Neat!

Compiling your Code...
> Success!
Running Test Cases...
> TestCase - Easy Success
> TestCase - Easy Success
> TestCase - Hard Success
Final Verdict
> Correct Answer

Test Test With Custom Input Submit